

编译器设计专题实验报告

实验一：DFA 模拟器实现

张凌曜 2234214014 计算机 2306

1 实验目的

用 C++ 写一个 DFA 模拟程序，能从文件读取 DFA 五元组定义，判断 DFA 合法性，枚举长度不超过 N 的所有合法字符串，以及模拟字符串识别过程。

2 实验原理

DFA 就是五元组 $(Q, \Sigma, \delta, q_0, F)$ ，分别是状态集、字符集、转移函数、初态和接受状态集。识别字符串时从初态出发，按输入字符逐步转移，最终落在接受状态就算接受。

枚举合法字符串我用了 BFS：从初态开始逐层扩展所有可能的字符组合，凡是到达接受状态的路径就对应一个合法字符串。

3 实验过程

3.1 DFA 状态图

测试用的 DFA 如图 1，用五元组写出来就是：

$$Q = \{1, 2, 3, 4\}, \quad \Sigma = \{a, b\}, \quad q_0 = 1, \quad F = \{3\}$$

转移函数 δ : $\delta(1, a) = 2$, $\delta(2, b) = 3$, $\delta(2, a) = 4$, $\delta(4, b) = 3$ 。

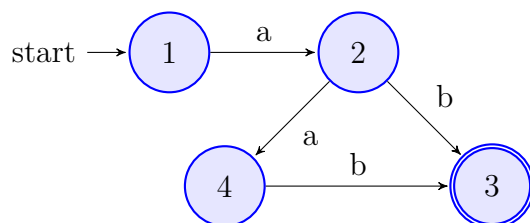


图 1: 测试 DFA 状态转换图

3.2 输入文件格式

DFA 五元组存在 `dfa_in1.dfa` 里，一行一类信息，比较直观：

```
1 a b          % 字符集
2 1 2 3 4      % 状态集
3 1            % 开始状态
4 3            % 接受状态集
5 1 a 2        % 转换表
6 2 b 3
7 2 a 4
8 4 b 3
```

3.3 实现要点

程序里用 `map<pair<string,char>, string>` 存转移表，查表方便。合法性检查就是验证初态在状态集里、接受状态集非空且包含于状态集。

字符串识别部分，逐字符查表做转移，同时把经过的状态记到 `vector` 里输出路径。BFS 枚举用的是队列，队列元素是当前字符串、当前状态和路径，每次取出一个元素尝试扩展所有字符，长度不超过 N 且到达接受状态就保存。

关于 BFS 的复杂度：每一层最多扩展 $|\Sigma|^k$ 个字符串（ k 是当前层深度），如果 DFA 里有环的话合法字符串数量会随 N 指数增长，队列的内存消耗也会跟着涨。本实验 N 取 3 所以没问题，但如果 N 比较大（比如 20 以上）就得考虑内存限制了。

4 测试结果

4.1 合法性检查与异常测试

程序启动后会自动打印 DFA 信息并进行合法性检查：

```

PS C:\Users\corey\Desktop\编译实验\第一次实验> .\dfa.exe
请输入DFA文件路径: dfa_in1.dfa
===== DFA 信息 =====
字符集: {a, b}
状态集: {1, 2, 3, 4}
开始状态: 1
接受状态集: {3}
状态转换表:
  1 --a--> 2
  2 --a--> 4
  2 --b--> 3
  4 --b--> 3
=====

===== DFA 合法性检查 =====
[检查] 开始状态: 1 - 合法
[检查] 接受状态集非空 - 合法
DFA 合法性检查通过!

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能: 1
请输入最大长度 N: 3

===== 长度≤3的合法字符串 =====
ab -> 1,2,3
aab -> 1,2,4,3
共 2 个合法字符串

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能: 2
请输入待判定的字符串: ab

===== DFA 识别结果 =====
输入字符串: ab
状态序列: 1,2,3
判定结果: 接受 (合法字符串)

```

除了正常识别之外，也测了几个异常情况：

异常输入	程序行为
初态不在状态集中（如 $q_0 = 5$ ）	报错：开始状态不合法
接受状态不在状态集中	报错：接受状态集包含非法状态
转移表含字符集外的字符	该条转移被忽略，不影响运行

4.2 字符串识别

输入 ab 时状态从 1 经 a 到 2 再经 b 到 3（接受状态），没问题。输入 a 停在状态 2 不是接受状态，拒绝。输入 b 第一步就没有转移，直接拒绝。

表 1: 字符串识别结果

输入	状态序列	结果
ab	1,2,3	接受
a	1,2	拒绝
aab	1,2,4,3	接受
b	—	拒绝

枚举最大长度 $N = 3$ 时输出 `ab` 和 `aab` 两个字符串，手算验证过确实就这两个。

```

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能：2
请输入待判定的字符串：aab

===== DFA 识别结果 =====
输入字符串：aab
状态序列：1,2,4,3
判定结果：接受（合法字符串）

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能：2
请输入待判定的字符串：a

===== DFA 识别结果 =====
输入字符串：a
状态序列：1,2
判定结果：拒绝（非法字符串）

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能：2
请输入待判定的字符串：b

===== DFA 识别结果 =====
输入字符串：b
状态序列：1
判定结果：拒绝（非法字符串）

===== 功能菜单 =====
1. 输出长度≤N的所有合法字符串
2. 判定输入字符串是否被DFA接受
3. 退出
请选择功能：

```

5 实验总结

写 BFS 枚举的时候踩了个坑：一开始用 `const auto&` 引用队列里的 `vector` 元素，结果 `push_back` 触发 `vector` 扩容后引用就悬空了，程序直接崩。改成值拷贝就没事了，C++ 容器迭代器失效这块确实得注意。

整体来说这个实验难度还行，文件输入格式设计比较直观，两个功能都能跑通。